

Reviewer Pack — Algebraic Topology Invariants and Boolean Circuit Monotonicity...

Entry ca450fbce567 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 00:23:05 UTC

Algebraic Topology Invariants and Boolean Circuit Monotonicity

Entry ID: ca450fbce567 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 00:23:05 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology **Field B** (complexity object): Boolean Circuit Monotonicity

Statement:

[‘For every boolean circuit C with n inputs, the Euler characteristic $\chi(C)$ of its configuration space is polynomially related to its monotone complexity $\mu(C)$. Specifically, $\chi(C) = \Theta(\mu(C)^k)$ for some constant k .’, ‘If a counterexample exists where $\chi(C) \neq \Theta(\mu(C)^k)$, then C cannot be realized by a constant-size monotone circuit.’, ‘For all circuits with $n \leq 40$ inputs, the conjecture holds if $\chi(C) = O(n^2)$ and $\mu(C) = O(n)$.’]

Rationale (proposer’s reasoning):

[‘Algebraic topology provides tools to study the topological properties of spaces. The Euler characteristic is a topological invariant that can be used to classify spaces up to homotopy equivalence. Boolean circuit monotonicity is a measure of how complex a circuit is, and it has been shown to be related to complexity classes. By connecting these two fields, we may uncover new insights into the nature of boolean circuits.’, ‘Previous research has demonstrated that certain topological invariants can bound computational problems like SAT refutation size. This suggests that algebraic topology could provide similar bounds for circuit monotonicity.’]

Taxonomy category: Algebraic Topology_Boolean Circuit Monotonicity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2ceaa1e8f3aa1434

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For circuits with $n \leq 40$ inputs, $\chi(C) = O(n^2)$ AND $\mu(C) = O(n)$, and $\chi(C)$ is polynomially related to $\mu(C)^k$ with k within $[1,5]$ for all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic topology AND boolean circuit monotonicity - Euler characteristic OR χ configuration space AND boolean circuit complexity - monotone complexity AND $\theta(n^2)$ AND $\theta(n)$

Top relevant hits considered: - [s2:2502.10590] Optimal Bounds for the Number of Pieces of Near-Circuit Hypersurfaces - [s2:10.1007/978-3-030-64378-2_23] A Secret-Sharing Based MPC Protocol for Boolean Circuits with Good Amortized Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        # Generate a random boolean circuit with n inputs
        return [random.choice([0, 1]) for _ in range(2**n)]

    def configuration_space(circuit):
        # Compute the configuration space of the circuit
        n = int(math.log2(len(circuit)))
        config_space = []
        for i in range(2**n):
            config = [int(x) for x in format(i, f'0{n}b')]
            if all(config[j] == circuit[2*j] or config[j] == circuit[2*j+1] for j in range(n)):
                config_space.append(config)
        return config_space

    def euler_characteristic(space):
        # Compute the Euler characteristic of the configuration space
        return len(space) - sum(len(list(group)) for _, group in itertools.groupby(sorted(space)))

    def monotone_complexity(circuit):
        # Compute the monotone complexity of the circuit
```

```

n = int(math.log2(len(circuit)))
max_depth = 0
visited = set()

def dfs(node, depth):
    nonlocal max_depth
    if node in visited:
        return
    visited.add(node)
    if depth > max_depth:
        max_depth = depth
    for i in range(n):
        if circuit[2*i] == node or circuit[2*i+1] == node:
            dfs(circuit[2*i], depth + 1)
            dfs(circuit[2*i+1], depth + 1)

dfs(0, 0)
return max_depth

n = random.randint(5, 40)
circuit = generate_circuit(n)
config_space = configuration_space(circuit)
chi = euler_characteristic(config_space)
mu = monotone_complexity(circuit)

if chi == 0 or mu == 0:
    return {
        "metric_name": "chi_over_mu",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "chi or mu is zero"
    }

chi_over_mu = chi / (mu ** 2)
return {
    "metric_name": "chi_over_mu",
    "metric_value": chi_over_mu,
    "instances_tested": 1,
    "conjecture_holds": chi_over_mu <= 10, # Arbitrary constant for testing
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='chi_over_mu > 10' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12813
2	preregistration	ollama_remote	glm4:latest	0	0	6086
3	novelty	ollama_remote	glm4:latest	0	0	4594
4	novelty	ollama_remote	glm4:latest	0	0	7348
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14331
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9196
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8719
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9915
9	judge	ollama_remote	glm4:latest	0	0	15674

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 88677 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/ca450fbce567.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ca450fbce567.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ca450fbce567.tar.gz` (if generated)